B26 Fine-tuning pretrained models

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2

B26 Fine-tuning pretrained models 3

B26-L1 Transfer learning and loading pretrained weights 3

B26-L2 Learning rates for fine-tuning, warm-up and decay 8

B26-L3 LoRA, fine-tuning with a low-rank update 14

B26-L4 Fine-tuning a text classifier with Hugging Face 19

B26-L5 Token classification, named entities and subword labels 26

B26-L6 Evaluating a fine-tuned model 31

Track B best-practice checklist 37

Appendix: solutions and answer keys 38

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B26 Fine-tuning pretrained models

Roadmap: Months 10 to 12: Transformers applied. Loading pretrained weights, learning-rate warm-up, LoRA, Hugging Face classification and token tagging, and evaluating a fine-tuned model.

B26-L1 Transfer learning and loading pretrained weights

Concept

A pretrained model has learned from a large amount of unlabelled data. **Fine-tuning** starts from its weights and trains on your few labelled examples. In the B26 lab, an encoder pretrained on 20,000 unlabelled sequences reached 0.992 accuracy from 60 labels after full fine-tuning. The same model trained from random weights reached 0.759.

Weights travel as a **state_dict**: a dictionary from parameter names to tensors. In PyTorch 2.15:

```
1 torch.save(model.state_dict(), "encoder.pt")
2 state = torch.load("encoder.pt")
    # weights_only=True by default since 2.6
3 result = model.load_state_dict(state, strict=False)
4 result.missing_keys, result.unexpected_keys
```

Three behaviours of `load_state_dict` matter when you fine-tune:

- With the default `strict=True`, any missing or extra key raises `RuntimeError`.
- With `strict=False`, it loads the keys that match and returns `missing_keys` and `unexpected_keys`. If every name carries an extra prefix such as `model.`, nothing matches. Every key is reported, no weight loads, and no error is raised. Always read the result.
- A key whose shape differs raises, even with `strict=False`. A checkpoint head with 2 outputs does not fit a new head with 3. Drop the old head's keys before loading.

The new head starts at random. Its keys appear in `missing_keys`, and that is expected. Any other missing key means the backbone did not load.

Then choose what trains. A frozen backbone with a new head, as in lesson B24-L5, is cheapest: call `p.requires_grad_(False)` on each backbone parameter. Full fine-tuning moves every weight, at a smaller learning rate.

Lesson B26-L3 adds a third option, LoRA. Count what trains with `sum(p.numel() for p in model.parameters() if p.requires_grad)`.

Why it matters

A team fine-tuned a pretrained text encoder on 500 coded survey answers and got accuracy no better than a model trained from scratch. The checkpoint had been saved from a wrapper module, so every key began with `model.`. The loading code used `strict=False` and ignored the result. No pretrained weight had loaded. Checking `missing_keys` would have shown the problem before the first epoch.

Worked example: weak code

```
1 state = torch.load("pretrained.pt")
2 model.load_state_dict(state, strict=False)
3 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
```

1. **Line 1.** The checkpoint's own head is still in state. If its shape differs from the new head, loading raises.
2. **Line 2.** The result is ignored. If no key matched, the model trains from random weights and nothing warns you.

Worked example: strong code

```
1  import torch
2  from torch import nn
3
4  BACKBONE_PREFIX = "encoder."
5
6
7  def load_backbone(model: nn.Module, checkpoint: dict[str, torch.Tensor]) -
   > list[str]:
8      """Copy the encoder weights from a pretraining checkpoint into a new
   model.
9
10     Args:
11         model: A model whose encoder sits under `encoder.`, with a new
   head.
12         checkpoint: A state_dict saved after pretraining.
13
14     Returns:
15         The model's keys the checkpoint did not fill, sorted: the new
   head.
16
17     Raises:
18         ValueError: If the checkpoint has no encoder keys, or the encoder
   did not load fully.
19     """
```

```
    backbone = {k: v for k, v in checkpoint.items() if
k.startswith(BACKBONE_PREFIX)}
21    if not backbone:
22        raise ValueError("checkpoint has no keys starting with
'encoder.'")
    result = model.load_state_dict(backbone, strict=False)
24    missing = [k for k in result.missing_keys if
k.startswith(BACKBONE_PREFIX)]
    if missing or result.unexpected_keys:
26        raise ValueError(f"encoder did not load: missing {missing},
unexpected {result.unexpected_keys}")
    return sorted(result.missing_keys)
```

1. **Line 20.** Keep the encoder only. The old head would raise a size mismatch.
2. **Line 23.** strict=False because the new head is meant to be missing.
3. **Line 25.** Read the result: any missing encoder key or unexpected key means the weights did not load.
4. **Line 27.** Return what stayed random, so the caller can log it.

Load only the backbone, use strict=False for the new head, and check missing_keys and unexpected_keys every time.

Recap card

- **state_dict** A dictionary from parameter names to tensors. torch.save writes it, torch.load reads it (weights_only=True by default since PyTorch 2.6).
- **load_state_dict(strict=False)** Loads matching names and returns missing_keys and unexpected_keys. Nothing raises if no name matches, so always check the result.
- **Shape mismatch** Raises even with strict=False. Drop the old head's keys before loading into a model with a new head.
- **Expected missing keys** Only the new head. Any other missing key means the backbone did not load.

Practice

Exercise 1. Multiple choice (level 1)

Every key in a checkpoint starts with `model.`, but your model's keys do not. What does `model.load_state_dict(state, strict=False)` do?

- a) Loads no weights, raises no error, and lists every key in missing_keys or unexpected_keys
- b) Raises RuntimeError because the names differ
- c) Strips the prefix and loads every weight

- d) Loads the weights whose shapes match, ignoring the names

Why?

- r1) strict=False matches keys by exact name and reports the rest instead of raising.
- r2) strict=False only skips the shape checks.

Exercise 2. Predict the output (level 2)

A pretrained network had a 2-class head. You build the same network with a 3-class head and load only the body. What does this print?

```
import torch
from torch import nn

pretrained = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 2))
state = {k: v for k, v in pretrained.state_dict().items() if
    k.startswith("0.")}
model = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 3))
result = model.load_state_dict(state, strict=False)
print(result.missing_keys, result.unexpected_keys)
```

Exercise 3. Fill the gap (level 2)

Load a backbone where the new head is meant to be missing, then stop if any key from the checkpoint went unused.

```
result = model.load_state_dict(backbone, strict=_____(1))
if result._____(2):
    raise ValueError(f"unused checkpoint keys: {result.unexpected_keys}")
```

Exercise 4. Write the function (level 3)

Write `compare_keys(model_keys, checkpoint_keys) -> tuple[list[str], list[str]]`, the check `load_state_dict(strict=False)` reports.

- `missing`: model keys not in the checkpoint. `unexpected`: checkpoint keys not in the model.
- Return both sorted, as `(missing, unexpected)`.

What you need to know.

Set difference keeps the items of the first set that the second lacks:

```
{"a", "b", "c"} - {"b"}          # {'a', 'c'}
sorted({"c", "a"})               # ['a', 'c']
```

Your code must pass these checks:

- Missing keys are the model's keys that the checkpoint lacks, sorted.
- Unexpected keys are the checkpoint's keys that the model lacks, sorted.
- When every name matches, both lists are empty.

- Return both lists sorted. Set order is arbitrary.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Keys a strict=False load would report.
def compare_keys(
    model_keys: list[str],
    checkpoint_keys: list[str],
) -> tuple[list[str], list[str]]: ...
```

Starter code:

```python
def compare_keys(model_keys, checkpoint_keys):
    return [], []
```

Exercise 5. Write the function (level 3)

Checkpoints saved from a wrapper module carry a prefix such as `model.` on every key. Write `strip_prefix(state, prefix) -> dict`.

- Return a new dictionary with `prefix` removed from the start of every key, values unchanged.
- Raise `ValueError` if any key does not start with `prefix`, so a wrong prefix fails loudly.

What you need to know.

`str.removeprefix` drops a prefix when it is there and leaves the string alone otherwise:

```python
"net.layer.w".removeprefix("net.")    # 'layer.w'
"layer.w".removeprefix("net.")        # 'layer.w'
```

Your code must pass these checks:

- Remove the prefix from the start of every key and keep the values.
- Remove the prefix only at the start, not wherever it appears.
- Raise ValueError when a key lacks the prefix.
- Return a new dictionary. Leave the input unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Remove a wrapper prefix from every checkpoint key.
def strip_prefix(
    state: dict[str, object],
    prefix: str,
) -> dict[str, object]: ...
```

Starter code:

```python
def strip_prefix(state, prefix):
    return state
```

Exercise 6. Spot the bug (level 4)

Fine-tuning from this checkpoint scores the same as training from scratch. The checkpoint's keys begin with `model.encoder.`. Tap the line that lets the problem pass unnoticed.

```
1  import torch
2
3
4  def build(model, path):
5      state = torch.load(path)
6      backbone = {k: v for k, v in state.items() if "encoder." in k}
7      model.load_state_dict(backbone, strict=False)
8      for p in model.encoder.parameters():
9          p.requires_grad_(False)
10     return model
```

Write down the line number of the bug.

Sources

- PyTorch 2.15: load_state_dict with strict=False returns _IncompatibleKeys(missing_keys, unexpected_keys); a prefixed checkpoint loads nothing and raises nothing; a size mismatch raises 'size mismatch for head.weight: copying a param with shape torch.Size([2, 8]) from checkpoint, the shape in current model is torch.Size([3, 8])' even with strict=False; strict=True reports 'Missing key(s) in state_dict'; torch.load refuses a pickled class by default with 'Weights only load failed'. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)
- B26 lab: with 60 labelled sequences, full fine-tuning from the pretrained encoder reached 0.992 mean validation accuracy over seeds 0 to 2, and training from scratch 0.759. (verified: labs/finetune reference run, PyTorch 2.15.0, November 2026.)

B26-L2 Learning rates for fine-tuning, warm-up and decay

Concept

Fine-tuning uses a smaller learning rate than training from scratch, because the weights already work. Hugging Face's `TrainingArguments` defaults to `learning_rate=5e-5` (transformers 5.22).

The first steps are the riskiest. The new head is random, so its gradients are large and noisy, and AdamW's running averages start from few samples. A

full-size step at that point can undo what pretraining learned. **Warm-up** starts the rate at 0 and raises it in a straight line over the first steps. **Linear decay** then lowers it to 0 by the last step:

```
def warmup_linear(step, warmup_steps, total_steps):
    if step < warmup_steps:
        return step / warmup_steps
    return max(0.0, (total_steps - step) / (total_steps - warmup_steps))
```

The function returns a factor. The rate at each step is the base rate times the factor. With 4 warm-up steps out of 10 and a base rate of 0.001, the rates are 0, 0.00025, 0.0005, 0.00075, 0.001, then down by 0.000167 a step to 0.

In PyTorch 2.15, `torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)` multiplies the base rate by `lr_lambda(step)`, and `scheduler.get_last_lr()` shows the current rate. transformers 5.22 builds the same schedule with `get_linear_schedule_with_warmup(optimizer, num_warmup_steps, num_training_steps)`.

Two rules make it work:

- **Count steps, not epochs.** `num_training_steps` is batches per epoch times epochs. With 60 examples in batches of 16, that is 4 batches, so 60 epochs make 240 steps.
- **Step the scheduler after the optimiser, once per batch.** Call `scheduler.step()` before `optimizer.step()` and PyTorch warns. Call it once per epoch and a 100-step warm-up lasts 100 epochs.

In `TrainingArguments`, `warmup_steps` takes a count, or a share of all steps when given a float below 1. transformers 5 removed the older `warmup_ratio`.

Why it matters

A team fine-tuned a text classifier for 3 epochs of 100 batches, and validation accuracy barely moved from the untrained model. The schedule had 100 warm-up steps, but `scheduler.step()` ran once per epoch. After 3 epochs the schedule had taken 3 steps, so the rate was 3% of its target. Moving the call into the batch loop fixed it.

Worked example: weak code

```
scheduler = get_linear_schedule_with_warmup(optimizer,
    num_warmup_steps=100, num_training_steps=1000)
for epoch in range(10):
    for xb, yb in loader:
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
```

```
        optimizer.step()
    scheduler.step()
```

1. **Line 1.** 1,000 is a guess, not len(loader) * epochs.
2. **Line 8.** Once per epoch: after 10 epochs the schedule has taken 10 of its 1,000 steps and is still warming up.

Worked example: strong code

```
import torch


def warmup_linear(step: int, warmup_steps: int, total_steps: int) -> float:

    """Learning-rate factor: linear warm-up from 0 to 1, then linear decay to 0.

     Args:
         step: Optimiser steps taken so far, from 0.
         warmup_steps: Steps to rise from 0 to 1. Zero means start at 1.
         total_steps: Step at which the factor reaches 0.

     Returns:
         The factor in [0, 1].

     Raises:
         ValueError: If warmup_steps is negative or not below total_steps.
     """
     if warmup_steps < 0 or warmup_steps >= total_steps:
         raise ValueError(f"need 0 <= warmup_steps < total_steps, got {warmup_steps}, {total_steps}")
     if step < warmup_steps:
         return step / warmup_steps
     return max(0.0, (total_steps - step) / (total_steps - warmup_steps))


def make_scheduler(
    optimizer: torch.optim.Optimizer, batches_per_epoch: int, epochs: int, warmup_share: float
) -> torch.optim.lr_scheduler.LambdaLR:
     """A per-batch schedule: warm-up over a share of all steps, then linear decay."""
     total = batches_per_epoch * epochs
     warmup = int(warmup_share * total)
     return torch.optim.lr_scheduler.LambdaLR(optimizer, lambda step: warmup_linear(step, warmup, total))
```

1. **Line 21.** Warm-up: 0 at step 0, 1 at the end of warm-up.
2. **Line 22.** Decay: 1 at the end of warm-up, 0 at the last step, never negative.
3. **Line 29.** Steps are batches, so the schedule needs the batch count.

4. **Line 31.** LambdaLR multiplies the base rate by the factor. Call scheduler.step() after optimizer.step() in the batch loop.

Warm up from 0, decay to 0, count steps as batches times epochs, and step the scheduler after the optimiser every batch.

Recap card

- **Warm-up** Raise the rate from 0 in a straight line over the first steps, so early noisy gradients do not damage pretrained weights.
- **Linear decay** From the end of warm-up, lower the rate in a straight line to 0 at the last step: (total - step) / (total - warmup).
- **Steps** Batches per epoch, rounded up, times epochs. Call scheduler.step() after optimizer.step() once per batch.
- **transformers 5.22** get_linear_schedule_with_warmup(optimizer, num_warmup_steps, num_training_steps); TrainingArguments learning_rate 5e-5, warmup_steps a count or a share below 1.

Practice

Exercise 1. Multiple choice (level 1)

Why does fine-tuning warm the learning rate up from 0 over the first steps?

- a) Early gradients from the random head are large and noisy, and full-size steps then can damage the pretrained weights
- b) The tokenizer needs time to build its vocabulary
- c) A rate of 0 makes the first epoch faster
- d) Warm-up replaces the need for a validation set

Why?

- r1) Small first steps let the head and the optimiser's averages settle before the weights move much.
- r2) Pretrained weights cannot change until the rate passes 1e-5.

Exercise 2. Predict the output (level 2)

What does this print?

```
def warmup_linear(step, warmup_steps, total_steps):
    if step < warmup_steps:
        return step / warmup_steps
    return max(0.0, (total_steps - step) / (total_steps - warmup_steps))


print([round(warmup_linear(s, 2, 8), 2) for s in range(0, 9, 2)])
```

Exercise 3. Fill the gap (level 2)

Size the schedule in steps and step it in the right place.

```
total_steps = len(loader) * _____(1)
for epoch in range(epochs):
    for xb, yb in loader:
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        scheduler._____(2)()
```

Exercise 4. Write the function (level 3)

Write `warmup_linear(step, warmup_steps, total_steps) -> float`.

- For `step < warmup_steps`, return `step / warmup_steps`.
- After that, return `(total_steps - step) / (total_steps - warmup_steps)`, never below 0.
- Raise `ValueError` unless `0 <= warmup_steps < total_steps`.

Your code must pass these checks:

- Rise from 0 to 1 in a straight line over the warm-up steps.
- Decay from 1 to 0 between the end of warm-up and total_steps, and stay at 0 after.
- With no warm-up, start at 1.
- Raise ValueError unless 0 <= warmup_steps < total_steps.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Learning-rate factor: linear warm-up from 0 to 1, then linear decay to 0.
def warmup_linear(
    step: int,
    warmup_steps: int,
    total_steps: int,
) -> float: ...
```

Starter code:

```
def warmup_linear(step, warmup_steps, total_steps):
    return 1.0
```

Exercise 5. Write the function (level 3)

Write `schedule_steps(n_examples, batch_size, epochs, warmup_share) -> tuple[int, int]` returning `(total_steps, warmup_steps)`.

- Batches per epoch round up: 60 examples in batches of 16 make 4 batches.
- `total_steps` is batches per epoch times epochs. `warmup_steps` is `int(warmup_share * total_steps)`.
- Raise `ValueError` if `n_examples`, `batch_size` or `epochs` is below 1, or `warmup_share` is outside 0 to 1, 1 excluded.

What you need to know.

`math.ceil` rounds up to the next whole number:

```
import math
math.ceil(7 / 2)     # 4
math.ceil(6 / 2)     # 3
```

Your code must pass these checks:

- Round batches per epoch up: a last, smaller batch is still a step.
- When the batch size divides evenly, there is no extra batch.
- Raise ValueError for counts below 1 or a warm-up share outside [0, 1).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Total optimiser steps and warm-up steps for a run.
def schedule_steps(
    n_examples: int,
    batch_size: int,
    epochs: int,
    warmup_share: float,
) -> tuple[int, int]: ...
```

Starter code:

```
def schedule_steps(n_examples, batch_size, epochs, warmup_share):
    return n_examples // batch_size * epochs, 0
```

Exercise 6. Spot the bug (level 4)

The learning rate is still rising when this run ends after 5 epochs of 50 batches. Tap the line responsible.

```
total = len(loader) * epochs
scheduler = get_linear_schedule_with_warmup(optimizer, int(0.1 * total),
    total)
for epoch in range(epochs):
    model.train()
    for xb, yb in loader:
```

```
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    scheduler.step()
```

Write down the line number of the bug.

Sources

- PyTorch 2.15: LambdaLR(optimizer, lr_lambda, last_epoch=-1) sets the rate to the base rate times lr_lambda(step), from step 0 at construction; calling scheduler.step() before optimizer.step() warns 'Detected call of `lr_scheduler.step()` before `optimizer.step()`'. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)
- transformers 5.22.1: get_linear_schedule_with_warmup(optimizer, num_warmup_steps, num_training_steps, last_epoch=-1) gives 0, 0.00025, 0.0005, 0.00075, 0.001, 0.000833 ... 0 for 4 warm-up steps of 10 at base rate 0.001; stepping it once per epoch for 3 epochs of 100 batches with 100 warm-up steps leaves the rate at 3e-05. TrainingArguments defaults: learning_rate 5e-05, weight_decay 0.0, num_train_epochs 3.0, warmup_steps 0 (int, or float below 1 as a share), lr_scheduler_type linear; warmup_ratio and evaluation_strategy are not parameters. (verified: transformers 5.22.1 run locally and TrainingArguments signature and docstring inspected, November 2026.)

B26-L3 LoRA, fine-tuning with a low-rank update

Concept

Full fine-tuning changes every weight and stores a full copy per task. **LoRA** (low-rank adaptation) freezes a layer's weight `W`, shape `(d_out, d_in)`, and learns a small change made of two matrices: `A`, shape `(r, d_in)`, and `B`, shape `(d_out, r)`, with a small rank `r`:

```
y = x @ W.T + (alpha / r) * (x @ A.T @ B.T)
```

The product `B @ A` has the shape of `W` but at most rank `r`. `alpha / r` scales the update, so you can change `r` without retuning the learning rate.

- **Few values to train.** A 768 by 768 layer holds 589,824 weights. With `r = 8`, `A` and `B` hold `8 * (768 + 768) = 12,288`, about 2%.

- **B starts at zero.** `A` starts random and `B` at zero, so `B @ A` is zero and the adapted model starts identical to the pretrained one. peft 0.21 does this by default.
- **Merge for prediction.** After training, fold the update in: `W_merged = W + (alpha / r) * B @ A`. The merged layer is an ordinary linear layer with no extra cost. Keep `A` and `B` apart instead, and one frozen base model serves many tasks, each with its own small adapter.
- **Where to put it.** A common choice is the attention query and value projections.

In the B26 lab, LoRA with `r = 4` on queries and values, plus a new head, trained 1,123 of 26,979 values. On 60 labels it reached 0.967 mean accuracy over three seeds, against 0.992 for full fine-tuning. Its seeds also spread wider, from 0.929 to 0.990. On 600 labels the gap closed to 0.995 against 0.998.

LoRA does not make the forward pass cheaper during training: the frozen weight still runs. It saves the memory for gradients and optimiser state of the frozen weights, and the storage of one full model copy per task.

Why it matters

A team fine-tuned one text encoder for each of 12 survey clients, each a full copy of 110 million weights. Storage and loading time grew with every client. Switching to LoRA adapters kept one frozen base model and 12 small adapter files, and each client's accuracy stayed within a point of its fully fine-tuned model.

Worked example: weak code

```
A = rng.normal(size=(r, d_in))
B = rng.normal(size=(d_out, r))
y = x @ W.T + alpha * (x @ A.T @ B.T)
```

1. **Line 2.** B random: the adapted layer differs from the pretrained one before any training.
2. **Line 3.** No division by r, so changing r changes the size of the update.

Worked example: strong code

```
import numpy as np


def lora_init(d_in: int, d_out: int, r: int, seed: int) ->
    tuple[np.ndarray, np.ndarray]:
    """A random, B zero, so the update starts at zero.
```

```
6
7       Raises:
8           ValueError: If r is below 1 or larger than d_in or d_out.
9       """
10      if not 1 <= r <= min(d_in, d_out):
11          raise ValueError(f"r must be in 1..{min(d_in, d_out)}, got {r}")
12      rng = np.random.default_rng(seed)
(1)     return rng.normal(0.0, 1.0 / np.sqrt(d_in), size=(r, d_in)),
        np.zeros((d_out, r))
14
15
16  def lora_forward(x: np.ndarray, W: np.ndarray, A: np.ndarray, B:
        np.ndarray, alpha: float) -> np.ndarray:
17      """Frozen layer plus the scaled low-rank update, for x of shape (n,
        d_in)."""
18      r = A.shape[0]
(2)     return x @ W.T + (alpha / r) * (x @ A.T @ B.T)
20
21
22  def merge_lora(W: np.ndarray, A: np.ndarray, B: np.ndarray, alpha: float)
        -> np.ndarray:
23      """One weight matrix equal to the frozen layer plus its update."""
(3)     return W + (alpha / A.shape[0]) * B @ A
```

1. **Line 13.** A random and B zero: B @ A starts at zero, so the model starts as pretrained.
2. **Line 19.** x @ A.T @ B.T never builds the d_out by d_in product, only (n, r) then (n, d_out).
3. **Line 24.** Merging uses the same alpha / r scale, so merged and unmerged outputs match.

LoRA freezes W and trains A and B with B at zero, scales the update by alpha / r, and merges into one matrix after training.

Recap card

- **LoRA output** y = x @ W.T + (alpha / r) * x @ A.T @ B.T, with W frozen, A (r, d_in), B (d_out, r).
- **LoRA starting point** A random, B zero, so the update is zero and training starts from the pretrained model.
- **LoRA size** r * (d_in + d_out) values per layer: 12,288 instead of 589,824 for 768 by 768 at r = 8.
- **Merging** W + (alpha / r) * B @ A as a new matrix: one plain layer, no extra cost at prediction.

Practice

Exercise 1. Multiple choice (level 1)

Why does LoRA start `B` at zero?

- a) So B @ A is zero and the adapted model starts identical to the pretrained one
- b) So the update has rank zero for the whole of training
- c) To halve the number of trainable values
- d) So that A receives no gradient

Why?

- r1) Training then starts from the pretrained model's behaviour and moves away only as far as the data asks.
- r2) Zero matrices take no memory.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import numpy as np

d_in, d_out, r = 6, 4, 2
W = np.ones((d_out, d_in))
A = np.full((r, d_in), 0.5)
B = np.zeros((d_out, r))
x = np.ones(d_in)
print(A.size + B.size, W.size, (x @ W.T + x @ A.T @ B.T == x @ W.T).all())
```

Exercise 3. Fill the gap (level 2)

Merge a trained LoRA update into the frozen weight.

```python
r = A.shape[0]
W_merged = W + (alpha / _____(1)) * _____(2) @ A
```

Exercise 4. Write the function (level 3)

Write `lora_forward(x, W, A, B, alpha) -> np.ndarray`.

- `x` is `(n, d_in)`, `W` is `(d_out, d_in)`, `A` is `(r, d_in)` and `B` is `(d_out, r)`.
- Return `x @ W.T + (alpha / r) * (x @ A.T @ B.T)`, with `r` the number of rows of `A`.
- Raise `ValueError` if `A` and `W` differ in width, or `B` is not `(d_out, r)`.

Your code must pass these checks:

- With B at zero, the output equals the frozen layer's, x @ W.T.
- Scale the update by alpha / r, where r is A's number of rows.

- Raise ValueError when A's width differs from W's, or B is not (d_out, r).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Frozen layer plus the scaled low-rank update.
def lora_forward(
    x: np.ndarray,
    W: np.ndarray,
    A: np.ndarray,
    B: np.ndarray,
    alpha: float,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def lora_forward(x, W, A, B, alpha):
    return x @ W.T
```

Exercise 5. Write the function (level 3)

Write `merge_lora(W, A, B, alpha) -> np.ndarray`, the weight of one plain layer that gives the same outputs as the frozen layer plus its LoRA update.

- Return `W + (alpha / r) * B @ A` as a new array, with `r` the number of rows of `A`.
- Leave `W` unchanged.

Your code must pass these checks:

- Return a new array. Do not add the update into W in place.
- x @ merged.T must equal x @ W.T + (alpha / r) * x @ A.T @ B.T.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One weight matrix equal to the frozen layer plus its LoRA update.
def merge_lora(
    W: np.ndarray,
    A: np.ndarray,
    B: np.ndarray,
    alpha: float,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def merge_lora(W, A, B, alpha):
    return W
```

Exercise 6. Spot the bug (level 4)

After merging, the model's predictions no longer match the unmerged adapter's. The run used `r = 4` and `alpha = 8`. Tap the line responsible, then fix it.

```
1 import numpy as np
2
3
4 def merged_weight(W, A, B, alpha):
5     r = A.shape[0]
6     scaling = alpha * r
7     update = B @ A
8     return W + scaling * update
```

Write down the line number of the bug. Then write the fix.

Sources

- peft 0.21.2: LoraConfig defaults r=8, lora_alpha=8, lora_dropout=0.0, init_lora_weights=True; the wrapped Linear has lora_A (r, d_in) and lora_B (d_out, r) with lora_B all zero, scaling lora_alpha / r (2.0 for r 8, alpha 16); outputs match the base model before training; merge_and_unload returns the base class with plain Linear layers and the same outputs. (verified: Run on peft 0.21.2 with transformers 5.22.1 and PyTorch 2.15.0, on a BERT model built from a config with random weights, November 2026.)
- B26 lab: LoRA with r 4 and alpha 8 on queries and values plus a new head trained 1,123 of 26,979 values; mean validation accuracy over seeds 0 to 2 was 0.967 (0.990, 0.982, 0.929) on 60 labels against 0.992 for full fine-tuning, and 0.995 against 0.998 on 600 labels. (verified: labs/finetune reference run, PyTorch 2.15.0, November 2026.)

B26-L4 Fine-tuning a text classifier with Hugging Face

Concept

Hugging Face transformers loads a pretrained model and its tokenizer by name. In transformers 5.22:

```
1 tokenizer = AutoTokenizer.from_pretrained(name)
2 model = AutoModelForSequenceClassification.from_pretrained(
3     name, num_labels=3, id2label=id2label, label2id=label2id)
4 batch = tokenizer(texts, padding=True, truncation=True, max_length=128,
    return_tensors="pt")
```

```
5 out = model(**batch, labels=labels)       # out.loss, out.logits
```

- **The load report.** Loading prints a table. `classifier.weight` and `classifier.bias` marked MISSING are the new head, initialised at random. That is expected. A missing encoder key is not (lesson B26-L1).
- **The batch.** `input_ids` holds token IDs, `attention_mask` holds 1 for real tokens and 0 for padding, and BERT-style models add `token_type_ids`. Pass them all with `**batch`.
- **The loss follows the labels.** The model sets `config.problem_type` from the first labels it sees. Integer labels give cross-entropy. Float labels with `num_labels` above 1 give a multi-label loss, sigmoid per class. `num_labels=1` gives mean squared error. One-hot float labels silently train a multi-label model.
- **Train mode.** `from_pretrained` returns the model in eval mode. Call `model.train()` before training, so dropout is on.

For LoRA, peft 0.21 wraps the model:

```
1 config = LoraConfig(task_type=TaskType.SEQ_CLS, r=8, lora_alpha=16,
    target_modules=["query", "value"])
2 model = get_peft_model(model, config)
3 model.print_trainable_parameters()
```

`task_type=TaskType.SEQ_CLS` keeps the new classifier trainable. Leave it out and the classifier stays frozen at its random start. On a small BERT, that trained 4,096 values instead of 4,291, and the missing 195 were the whole head. Target names depend on the architecture. BERT calls its projections `query` and `value`, and a name the model lacks raises `NoMatchingPeftModuleError`.

Why it matters

A team fine-tuned a LoRA classifier for complaint categories and got accuracy no better than always predicting the largest category. The training loss fell a little and stalled. Their `LoraConfig` had no `task_type`, so the randomly initialised head never trained, and the adapters could only bend the encoder towards a random projection. Adding `task_type=TaskType.SEQ_CLS` fixed it on the next run.

Worked example: weak code

```
1 model = AutoModelForSequenceClassification.from_pretrained(name,
    num_labels=3)
2 model = get_peft_model(model, LoraConfig(target_modules=["query",
    "value"]))
3 labels = torch.nn.functional.one_hot(torch.tensor(y), 3).float()
```

```
4 out = model(**batch, labels=labels)
```

1. **Line 1.** No id2label: predictions come back as LABEL_0, LABEL_1, LABEL_2.
2. **Line 2.** No task_type: the new classifier is frozen at random.
3. **Line 3.** Float labels switch the loss to multi-label, sigmoid per class, for a task with one label per text.

Worked example: strong code

```
1  import torch
2  from peft import LoraConfig, TaskType, get_peft_model
3  from transformers import AutoModelForSequenceClassification
4
5
6  def build_classifier(name: str, labels: list[str], r: int = 8) ->
     torch.nn.Module:
7      """A pretrained encoder with a new head and LoRA on queries and
     values.
8
9      Args:
10         name: Hugging Face model name or local folder.
11         labels: Class names, in ID order.
12         r: LoRA rank.
13
14     Returns:
15         The peft model in train mode.
16     """
1      id2label = dict(enumerate(labels))
18     label2id = {label: i for i, label in id2label.items()}
19     model = AutoModelForSequenceClassification.from_pretrained(
20         name, num_labels=len(labels), id2label=id2label, label2id=label2id
21     )
22     config = LoraConfig(
2          task_type=TaskType.SEQ_CLS, r=r, lora_alpha=2 * r,
     target_modules=["query", "value"]
24     )
25     peft_model = get_peft_model(model, config)
3      peft_model.train()
27     return peft_model
28
29
30 def training_loss(model: torch.nn.Module, batch: dict[str, torch.Tensor],
     labels: torch.Tensor) -> torch.Tensor:
31     """Cross-entropy loss for one batch. labels must be integer class
     IDs."""
4      if labels.dtype != torch.long:
33         raise TypeError(f"labels must be torch.long class IDs, got
     {labels.dtype}")
34     return model(**batch, labels=labels).loss
```

1. **Line 17.** Names travel with the saved model, so predictions read as class names.

2. **Line 23.** SEQ_CLS keeps the new head trainable. BERT names its projections query and value.
3. **Line 26.** from_pretrained returns eval mode. Training needs dropout on.
4. **Line 32.** Integer labels give cross-entropy. Floats would switch to a multi-label loss with no error.

Pass num_labels and the label maps, read the load report, use integer labels, call model.train(), and give LoraConfig a task_type.

Recap card

- **Loading a classifier (transformers 5.22)** AutoModelForSequenceClassification.from_pretrained(name, num_labels=..., id2label=..., label2id=...). The new head shows as MISSING in the load report.
- **Labels decide the loss** Integer IDs: cross-entropy. Floats with num_labels > 1: multi-label sigmoid. num_labels=1: mean squared error.
- **Tokenizer call** tokenizer(texts, padding=True, truncation=True, max_length=..., return_tensors="pt") gives input_ids and attention_mask.
- **peft LoRA for classification (0.21)** LoraConfig(task_type=TaskType.SEQ_CLS, target_modules=["query", "value"], ...). Without task_type the new head stays frozen.

Practice

Exercise 1. Multiple choice (level 1)

Loading a pretrained encoder with `AutoModelForSequenceClassification.from_pretrained(name, num_labels=4)` reports `classifier.weight` and `classifier.bias` as MISSING. What does it mean?

- a) The new 4-class head starts at random, as expected. Fine-tuning trains it
- b) The download failed and the model is unusable
- c) num_labels must match the pretraining task, so set it to 2
- d) The encoder weights are also random

Why?

- r1) The checkpoint was saved without a 4-class head, so the head has nothing to load from.
- r2) Hugging Face always resets heads to stop overfitting.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`BertConfig` describes a BERT model's size, and `BertForSequenceClassification(config)` builds one with random weights and no download. A tiny one for tests:

```
config = BertConfig(vocab_size=50, hidden_size=16, num_hidden_layers=1,
    num_attention_heads=2, intermediate_size=32)
model = BertForSequenceClassification(config)
```

```
import torch
from transformers import BertConfig, BertForSequenceClassification

config = BertConfig(vocab_size=30, hidden_size=32, num_hidden_layers=1,
                    num_attention_heads=4, intermediate_size=64,
    num_labels=3)
model = BertForSequenceClassification(config)
out = model(input_ids=torch.tensor([[2, 5, 6, 3]]),
    labels=torch.tensor([[0.0, 1.0, 0.0]]))
print(model.config.problem_type, tuple(out.logits.shape))
```

Exercise 3. Fill the gap (level 2)

Tokenise a batch of texts of different lengths into PyTorch tensors the model accepts.

```
batch = tokenizer(texts, padding=_____(1), truncation=True,
    max_length=128, return_tensors="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `label_maps(labels) -> tuple[dict[int, str], dict[str, int]]` for `from_pretrained(..., id2label=..., label2id=...)`.

- Use the distinct labels, sorted, so IDs do not depend on row order.
- Return `(id2label, label2id)`, with IDs from 0.
- Raise `ValueError` if there are fewer than 2 distinct labels.

What you need to know.

`enumerate` pairs each item with its position, and `dict` turns those pairs into a dictionary:

```
dict(enumerate(["x", "y"]))          # {0: 'x', 1: 'y'}
```

Your code must pass these checks:

- Number the distinct labels in sorted order, from 0.
- IDs must not depend on the order of the rows.

- Raise ValueError for fewer than 2 distinct labels.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# ID-to-label and label-to-ID maps for a classifier head.
def label_maps(
    labels: list[str],
) -> tuple[dict[int, str], dict[str, int]]: ...
```

Starter code:

```python
def label_maps(labels):
    return {}, {}
```

Exercise 5. Write the function (level 3)

Write `pad_batch(seqs, pad_id, max_length) -> tuple[list[list[int]], list[list[int]]]`, what `padding=True, truncation=True` does to token IDs.

- Cut each sequence to at most `max_length` IDs.
- Pad every sequence at the end with `pad_id` to the longest length after cutting.
- Return `(input_ids, attention_mask)`, with mask 1 for real IDs and 0 for padding.
- Raise `ValueError` if `seqs` is empty or `max_length` is below 1.

Your code must pass these checks:

- Pad at the end to the longest sequence, with mask 0 on padding.
- Cut to max_length first, then pad to the longest cut length.
- Build new lists. Do not pad the caller's lists in place.
- Raise ValueError for no sequences or max_length below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Truncate and right-pad token IDs, with an attention mask.
def pad_batch(
    seqs: list[list[int]],
    pad_id: int,
    max_length: int,
) -> tuple[list[list[int]], list[list[int]]]: ...
```

Starter code:

```python
def pad_batch(seqs, pad_id, max_length):
    return seqs, [[1] * len(s) for s in seqs]
```

Exercise 6. Spot the bug (level 4)

After fine-tuning, `model.classifier`'s weights are exactly as they were at the start, and accuracy is near chance. Tap the line responsible.

```
1  from peft import LoraConfig, get_peft_model
2  from transformers import AutoModelForSequenceClassification
3
4
5  def build(name, n_classes):
6      model = AutoModelForSequenceClassification.from_pretrained(name,
       num_labels=n_classes)
7      config = LoraConfig(r=8, lora_alpha=16, target_modules=["query",
       "value"])
8      model = get_peft_model(model, config)
9      model.train()
10     return model
```

Write down the line number of the bug.

Sources

- transformers 5.22.1: from_pretrained on a saved BertModel with num_labels=3 prints a load report listing classifier.weight and classifier.bias as MISSING and returns the model in eval mode; problem_type is None until the first labelled call, then single_label_classification for long labels (NllLoss), multi_label_classification for float labels with num_labels 3 (BCEWithLogits), regression for num_labels=1 (MSE); the tokenizer with padding=True returns input_ids, token_type_ids and attention_mask with 0 at padding. (verified: Run locally on transformers 5.22.1 and PyTorch 2.15.0 with a BERT built from a config and a local vocabulary, November 2026. No hub download: huggingface.co was blocked.)
- peft 0.21.2: get_peft_model with LoraConfig(r=8, lora_alpha=16, target_modules=['query', 'value'], task_type=TaskType.SEQ_CLS) on a 2-layer, 64-wide BERT with 3 labels trains 4,291 values, the classifier included via modules_to_save; without task_type it trains 4,096 and the classifier is frozen; print_trainable_parameters prints 'trainable params: 4,291 || all params: 110,534 || trainable%: 3.8821'; target_modules=['q_proj'] raises NoMatchingPeftModuleError. (verified: Run locally on peft 0.21.2, transformers 5.22.1 and PyTorch 2.15.0, November 2026.)

B26-L5 Token classification, named entities and subword labels

Concept

Named entity recognition (NER) labels every word. The usual scheme is **BIO**: `B-` begins an entity, `I-` continues it, and `O` is outside any entity. "Ana visited New York" is `B-PER O B-LOC I-LOC`. On open survey answers, NER finds brands, products and places.

The labels belong to words, but the model sees subword tokens. A BERT tokenizer splits "staffs" into `staff` and `##s`, and adds `[CLS]` and `[SEP]`. Pass the words already split, and ask which word each token came from (transformers 5.22):

```
enc = tokenizer(["Ana", "visited", "London", "staffs"],
    is_split_into_words=True)
enc.word_ids()     # [None, 0, 1, 2, 3, 3, None]
```

`None` marks a special token. A repeated ID marks a word split into several tokens. Align the word labels to tokens with three rules:

- a special token gets `-100`;
- the first token of a word gets the word's label;
- any later token of the same word gets `-100`.

`-100` is the default `ignore_index` of `nn.CrossEntropyLoss` in PyTorch 2.15, and the Hugging Face token-classification loss uses it. Those positions add nothing to the loss, so each word counts once, whatever its number of pieces.

`AutoModelForTokenClassification.from_pretrained(name, num_labels=len(tags), id2label=..., label2id=...)` returns logits of shape `(batch, tokens, num_labels)`. To predict, take the argmax per token, then read each word's label from its first token. The load report lists the pooler weights as UNEXPECTED. That is fine: a token head does not use the pooler.

Check alignment on a real example before training. Labels shifted by one position still train, and the loss still falls, but the model learns to tag the word next to each entity.

Why it matters

A team trained a model to find product names in open answers. Validation F1 looked fine on short answers and collapsed on answers with brand names such as "SuperSaver". Their alignment gave the word's label to every token

by position in the word list, so after the first split word, every label slid one or more tokens to the right. Aligning with `word_ids()` fixed it.

Worked example: weak code

```
enc = tokenizer(words, is_split_into_words=True)
labels = [-100] + word_labels + [-100]
labels = labels + [-100] * (len(enc["input_ids"]) - len(labels))
```

1. **Line 2.** Assumes one token per word. After the first split word, every label sits on the wrong token.
2. **Line 3.** Padding the list to the right length hides the misalignment instead of fixing it.

Worked example: strong code

```
IGNORE = -100


def align_labels(word_ids: list[int | None], word_labels: list[int]) -> list[int]:
    """Token labels from word labels: first token of each word only.

    Args:
        word_ids: From enc.word_ids(): the word index of each token, None for special tokens.
        word_labels: One label ID per word.

    Returns:
        One label per token, IGNORE for special tokens and later pieces of a word.

    Raises:
        ValueError: If a word index has no label.
    """
    aligned: list[int] = []
    previous: int | None = None
    for word_id in word_ids:
        if word_id is None:
            aligned.append(IGNORE)
        elif word_id != previous:
            if word_id >= len(word_labels):
                raise ValueError(f"word {word_id} has no label; got {len(word_labels)} labels")
            aligned.append(word_labels[word_id])
        else:
            aligned.append(IGNORE)
        previous = word_id
    return aligned


```

```
32 def encode_example(tokenizer, words: list[str], tags: list[str],
     label2id: dict[str, int]) -> dict[str, list[int]]:
33     """Token IDs, attention mask and aligned labels for one sentence."""
34     enc = tokenizer(words, is_split_into_words=True, truncation=True)
3      labels = align_labels(enc.word_ids(), [label2id[t] for t in tags])
36     return {"input_ids": enc["input_ids"], "attention_mask":
     enc["attention_mask"], "labels": labels}
```

1. **Line 20.** Special tokens. Use `is None`: word 0 is falsy and would be skipped by `if not word_id`.
2. **Line 22.** A new word starts: its first token carries the label.
3. **Line 35.** word_ids() comes from the same call, so it matches the tokens exactly, truncation included.

Tokenise with is_split_into_words=True, align with word_ids(), label the first piece of each word, and give everything else -100.

Recap card

- **BIO tags** B- begins an entity, I- continues it, O is outside. "New York" is B-LOC I-LOC.
- **word_ids() (transformers 5.22)** With is_split_into_words=True, the word index of each token: None for special tokens, repeats for split words.
- **Label alignment** None gets -100, a word's first token gets its label, later pieces get -100. Test with `is None`, never truthiness.
- **-100** The default ignore_index of nn.CrossEntropyLoss (PyTorch 2.15). Those positions add nothing to the loss.

Practice

Exercise 1. Multiple choice (level 1)

Why do special tokens and the later pieces of a split word get the label `-100`?

- a) Cross-entropy ignores -100 by default, so those positions add nothing to the loss
- b) -100 is the ID of the O tag
- c) The tokenizer cannot read -100, so it skips those tokens
- d) It tells the model to predict nothing at those positions at prediction time

Why?

- r1) Each word then counts once in the loss, whatever its number of tokens.
- r2) Negative labels make the gradient smaller.

Exercise 2. Predict the output (level 2)

What does this print?

```
word_ids = [None, 0, 1, 1, 2, None]
word_labels = [1, 0, 3]
aligned, previous = [], None
for w in word_ids:
    if w is None:
        aligned.append(-100)
    elif w != previous:
        aligned.append(word_labels[w])
    else:
        aligned.append(-100)
    previous = w
print(aligned)
```

Exercise 3. Fill the gap (level 2)

Complete the special-token rule in the alignment loop.

```
for word_id in enc.word_ids():
    if word_id is _____(1):
        labels.append(_____(2))
```

Exercise 4. Write the function (level 3)

Write `align_labels(word_ids, word_labels) -> list[int]`.

- A `None` word ID gets `-100`.
- The first token of each word gets that word's label. Later tokens of the same word get `-100`.
- Raise `ValueError` if a word ID has no label in `word_labels`.

Your code must pass these checks:

- Label the first token of each word, -100 for later pieces and special tokens.
- Word 0 is a real word. Test for None with `is None`, not truthiness.
- Raise ValueError when a word ID has no label.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Token labels from word labels, first token of each word only.
def align_labels(
    word_ids: list[int | None],
    word_labels: list[int],
) -> list[int]: ...
```

Starter code:

```
def align_labels(word_ids, word_labels):
```

```
    return [word_labels[w] for w in word_ids]
```

Exercise 5. Write the function (level 3)

Write `word_predictions(word_ids, token_preds) -> list[int]`: one predicted label per word, read from each word's first token.

- `token_preds` holds one predicted label per token, aligned with `word_ids`.
- Skip `None` positions. Return the predictions in word order.
- Raise `ValueError` if the two lists differ in length.

Your code must pass these checks:

- Take the prediction at each word's first token and skip special tokens.
- Word 0 counts. Compare with None using `is None`.
- Raise ValueError when word_ids and token_preds differ in length.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One label per word, from the word's first token.
def word_predictions(
    word_ids: list[int | None],
    token_preds: list[int],
) -> list[int]: ...
```

Starter code:

```python
def word_predictions(word_ids, token_preds):
    return token_preds
```

Exercise 6. Spot the bug (level 4)

The model never learns the first word of any sentence. Tap the line responsible, then fix it.

```python
def align_labels(word_ids, word_labels):
    aligned, previous = [], None
    for word_id in word_ids:
        if not word_id:
            aligned.append(-100)
        elif word_id != previous:
            aligned.append(word_labels[word_id])
        else:
            aligned.append(-100)
        previous = word_id
    return aligned
```

Write down the line number of the bug. Then write the fix.

Sources

- transformers 5.22.1: a BERT tokenizer with a vocabulary holding 'staff' and '##s' tokenises ['Ana', 'visited', 'London', 'staffs'] with is_split_into_words=True to [CLS] ana visited london staff ##s [SEP], word_ids() [None, 0, 1, 2, 3, 3, None]; BertForTokenClassification returns logits (1, 7, 5), and its load report from a saved BertModel lists pooler.dense weights as UNEXPECTED and classifier weights as MISSING. (verified: Run locally on transformers 5.22.1 and PyTorch 2.15.0 with a local vocabulary and a BERT built from a config, November 2026.)
- PyTorch 2.15: nn.CrossEntropyLoss().ignore_index is -100. (verified: Run on PyTorch 2.15.0, November 2026.)

B26-L6 Evaluating a fine-tuned model

Concept

A fine-tuned model's score means something only next to other numbers. Report at least three:

- **A trivial baseline**: always predict the most common class, or tag every word `O`.
- **A simple model**: for text, word counts with logistic regression. If fine-tuning does not beat it clearly, the cheaper model wins.
- **The other strategies**: from scratch, a frozen encoder with a new head, full fine-tuning and LoRA, as in the B26 lab.

Seeds. On small data, the result depends on the seed. In the lab, LoRA on 60 labels scored 0.990, 0.982 and 0.929 over three seeds. Run at least three seeds and report the mean and the range. Picking the best seed reports luck.

Validation, then test. Choose the strategy, learning rate and epochs on validation data. Score the test set once, at the end. Every extra look at the test set turns it into validation data.

Entity-level F1 for NER. Token accuracy flatters a tagger, because most tokens are `O`. If 90% of tokens are `O`, a model that tags everything `O` scores 0.90 token accuracy and finds no entity. Score whole entities instead. Turn each tag sequence into spans of type, start and end:

```
1 ["B-PER", "O", "B-LOC", "I-LOC"]    # {("PER", 0, 1), ("LOC", 2, 4)}
```

A predicted span counts only if type, start and end all match a true span. Precision is correct spans over predicted spans, recall is correct spans over

true spans, and F1 is their harmonic mean. Report it for each entity type as well, since a rare type hides inside the overall number.

Forgetting. Full fine-tuning overwrites the pretrained weights. If the same base model must serve other tasks, LoRA adapters leave it unchanged.

Why it matters

A team reported 0.96 accuracy for a product-name tagger on survey answers. A reviewer asked for entity-level F1: it was 0.41. Nine in ten tokens were `O`, and the model missed most multi-word product names or cut them short. The team had also tuned on the test set over twelve runs. A fresh test set and three seeds gave an honest 0.58, which set the plan for more labelling.

Worked example: weak code

```
scores = [run(seed) for seed in range(5)]
print("accuracy", max(scores))
token_acc = sum(p == t for p, t in zip(pred_tags, true_tags)) /
    len(true_tags)
```

1. **Line 2.** The best of five seeds reports luck, not the method.
2. **Line 3.** Token accuracy for NER: mostly measures how often O is right.

Worked example: strong code

```
def bio_spans(tags: list[str]) -> set[tuple[str, int, int]]:
    """Entity spans (type, start, end exclusive) from BIO tags.

    An I- tag after O or after another type starts a new entity.
    """
    spans: set[tuple[str, int, int]] = set()
    start: int | None = None
    kind: str | None = None
    for i, tag in enumerate([*tags, "O"]):
        continues = tag.startswith("I-") and tag[2:] == kind
        if not continues:
            if start is not None and kind is not None:
                spans.add((kind, start, i))
            start, kind = (i, tag[2:]) if tag != "O" else (None, None)
    return spans


def entity_f1(true_tags: list[str], pred_tags: list[str]) -> float:
    """F1 over whole entities: type, start and end must all match.

    Raises:
        ValueError: If the tag lists differ in length.
    """
```

```
    if len(true_tags) != len(pred_tags):
        raise ValueError(f"{len(true_tags)} true tags but {len(pred_tags)} predicted")
    true, pred = bio_spans(true_tags), bio_spans(pred_tags)
    if not true and not pred:
        return 1.0
    correct = len(true & pred)
    precision = correct / len(pred) if pred else 0.0
    recall = correct / len(true) if true else 0.0
    return 0.0 if precision + recall == 0 else 2 * precision * recall / (precision + recall)
```

1. **Line 9.** A trailing O closes an entity that runs to the end of the sentence.
2. **Line 10.** Only an I- tag of the same type continues the current entity.
3. **Line 29.** Set intersection: a span counts only if type, start and end all match.
4. **Line 32.** Guard the division when nothing is correct.

Compare against a trivial baseline and a simple model, report mean and range over seeds, score the test set once, and use entity-level F1 for NER.

Recap card

- **Baselines for a fine-tuned model** The trivial baseline (majority class, or all O), a simple model, and the other fine-tuning strategies, on the same validation set.
- **Seeds** Run at least three, report mean and range. The best seed reports luck.
- **Entity-level F1** Spans (type, start, end) must match exactly. Precision over predicted spans, recall over true spans.
- **Test set** Score it once, at the end. Every extra look turns it into validation data.

Practice

Exercise 1. Multiple choice (level 1)

A tagger scores 0.90 token accuracy on survey answers where 90% of tokens are `0`. What do you know about how well it finds entities?

- a) Nothing yet: tagging every token O would also score 0.90. Compute entity-level F1
- b) It finds about 90% of entities
- c) It is better than a baseline, because 0.90 is high
- d) It finds no entities

Why?

- r1) The score matches what the all-O baseline gets, so it says nothing about entities.
- r2) Token accuracy is always 0.1 higher than entity F1.

Exercise 2. Predict the output (level 2)

What does this print?

```
tags = ["B-PER", "O", "B-LOC", "I-LOC", "O", "B-ORG"]
spans, start, kind = [], None, None
for i, tag in enumerate(tags + ["O"]):
    if not (tag.startswith("I-") and tag[2:] == kind):
        if start is not None:
            spans.append((kind, start, i))
        start, kind = (i, tag[2:]) if tag != "O" else (None, None)
print(spans)
```

Exercise 3. Put in order (level 2)

Order the steps of an honest evaluation of a fine-tuned classifier.

- Score the majority-class baseline and a simple model on validation
- Score the final set-up once on the test set
- Split into training, validation and test sets, and set the test set aside
- Rerun the chosen set-up with three seeds and record mean and range
- Fine-tune and choose strategy, learning rate and epochs on validation

Exercise 4. Write the function (level 3)

Write `bio_spans(tags) -> set[tuple[str, int, int]]`: the entities in a BIO tag list as `(type, start, end)`, with `end` exclusive.

- `B-X` starts an entity of type `X`. `I-X` continues an entity of type `X`.
- An `I-X` after `O`, or after an entity of another type, starts a new entity.
- `O` ends any open entity, and so does the end of the list.

Your code must pass these checks:

- B- starts an entity, I- of the same type continues it, O ends it.
- Close an entity that runs to the end of the list.
- A B- tag always starts a new entity, and an I- tag of another type does too.
- An I- tag after O starts an entity. No tags give no spans.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Entity spans (type, start, end exclusive) from BIO tags.
def bio_spans(tags: list[str]) -> set[tuple[str, int, int]]: ...
```

Starter code:

```
def bio_spans(tags):
    return set()
```

Exercise 5. Write the function (level 3)

Write `entity_f1(true_tags, pred_tags) -> float`. `bio_spans(tags)` is defined for you and returns a set of `(type, start, end)` spans.

- Precision is correct spans over predicted spans. Recall is correct spans over true spans. F1 is `2 * p * r / (p + r)`.
- Return 1.0 when neither list has an entity, and 0.0 when nothing is correct.
- Raise `ValueError` if the lists differ in length.

Your code must pass these checks:

- A span cut short is wrong: 1 of 2 predicted and 1 of 2 true spans are correct, F1 0.5.
- Predicting no entities when there is one scores 0.0, not token accuracy.
- With no entities on either side, return 1.0.
- Raise ValueError when the lists differ in length.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# F1 over whole entities: type, start and end must all match.
def entity_f1(
    true_tags: list[str],
    pred_tags: list[str],
) -> float: ...
```

Starter code:

```
def entity_f1(true_tags, pred_tags):
    return sum(t == p for t, p in zip(true_tags, pred_tags)) /
    len(true_tags)
```

Exercise 6. Architecture design (level 4)

Plan the evaluation of a LoRA tagger for brand names, fine-tuned on 800 labelled survey answers.

Options: Entity-level F1, overall and for each entity type, Token accuracy, Final training loss.

1. Headline metric
2. Compare against
3. Seeds

Practical: Compare fine-tuning strategies, then fine-tune a model on your own task

Platform: Laptop. Work in `labs/finetune/` in this repository. You pretrain a small transformer on unlabelled sequences, then fine-tune it on 60 labels four ways: from scratch, a frozen encoder with a new head, full fine-tuning and LoRA. You write the weight loading, the freezing, the warm-up schedule and LoRA, and the tests check each. You need PyTorch 2.15 on the laptop. Then apply the same steps with Hugging Face to a work task. That fine-tuned model is the roadmap's evidence for month 12.

1. Run `cd labs/finetune`, create a virtual environment, then `pip install -e .` and `pip install pytest`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/finetune_lab/`.
2. Write `warmup_linear` in `schedule.py` (lesson B26-L2). Run `pytest tests/test_data_and_schedule.py`.
3. Write `load_backbone`, `set_trainable` and `count_parameters` in `strategies.py` (lesson B26-L1), then `LoRALinear` and `add_lora` in `lora.py` (lesson B26-L3). Run `pytest`: every test passes.
4. Run `python -m finetune_lab pretrain`, then `python -m finetune_lab compare` and `python -m finetune_lab compare --n-train 600`. Compare your tables with the README's reference numbers.
5. Choose a work task with a few hundred labelled texts: coded open answers for classification, or brand names for tagging. Fine-tune a small pretrained encoder from the Hugging Face Hub with transformers and peft (lessons B26-L4 and B26-L5). Use integer labels and `task_type`, and check the load report.
6. Evaluate as in this lesson: a majority-class baseline, a simple model, LoRA against full fine-tuning, three seeds, entity-level F1 if you tag. Score the test set once.
7. Write your README: the lab tables, your task, the data, every score with its range, and what you would try next. Push it to a new GitHub repository.

Sources

- B26 lab: LoRA with r 4 on 60 labels scored 0.990, 0.982 and 0.929 validation accuracy over seeds 0, 1 and 2. (verified: labs/finetune reference run, PyTorch 2.15.0, November 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B26-L1 Transfer learning and loading pretrained weights

B26-L1 exercise 1

Hints. (1) How does load_state_dict match a tensor to a parameter? (2) What does strict=False change? (3) Unmatched names are reported in the result.

Answer. a) Loads no weights, raises no error, and lists every key in missing_keys or unexpected_keys Because: r1) strict=False matches keys by exact name and reports the rest instead of raising.

Names are matched exactly. With `strict=False`, unmatched names go into `missing_keys` and `unexpected_keys` and nothing raises, so the model keeps its random weights unless you read the result.

Common wrong answers.

- `b`: That is strict=True. With strict=False, unmatched names are reported, not raised.
- `c`: PyTorch never rewrites names. You strip a prefix yourself.
- `d`: Loading matches by name only. A shape mismatch on a matching name raises even with strict=False.
- `a|r2`: Right answer, wrong reason. A shape mismatch still raises with strict=False. It relaxes the key-name check.

B26-L1 exercise 2

Hints. (1) nn.Sequential names parameters by position: 0, 1, 2. (2) Which keys does state hold? (3) Which of the model's keys does state not fill?

Answer.

```
1 ['2.weight', '2.bias'] []
```

`state` holds `0.weight` and `0.bias` only. The model's head, `2.weight` and `2.bias`, is missing and keeps its random start, which is what you want. Nothing is unexpected.

Common wrong answers.

- `[] []`: The new head's keys were left out of state, so they are missing.
- `['2.weight', '2.bias'] ['2.weight', '2.bias']`: Only keys starting with '0.' were kept, so the checkpoint has no head keys to be unexpected.

B26-L1 exercise 3

Hints. (1) The new head will not be in the checkpoint. (2) Which list holds checkpoint keys the model has no place for? (3) `False` and `unexpected_keys`.

Answer.

```
result = model.load_state_dict(backbone, strict=False)
if result.unexpected_keys:
    raise ValueError(f"unused checkpoint keys: {result.unexpected_keys}")
```

`strict=False` lets the new head stay missing. `unexpected_keys` lists checkpoint keys with no match in the model, a sign of a prefix or architecture mismatch.

Common wrong answers.

- g1 = `True`; g2 = `unexpected_keys`: strict=True raises on the missing head keys before you can check anything.
- g1 = `False`; g2 = `missing_keys`: The new head is always missing, so this raises every time. Unused checkpoint keys are unexpected_keys.

B26-L1 exercise 4

Hints. (1) Turn both lists into sets. (2) Missing: model set minus checkpoint set. (3) Sort each result.

Answer.

```
def compare_keys(model_keys: list[str], checkpoint_keys: list[str]) ->
    tuple[list[str], list[str]]:
    """Keys a strict=False load would report.

    Args:
        model_keys: Parameter names of the model.
        checkpoint_keys: Names in the checkpoint.

    Returns:
        (missing, unexpected), each sorted.
    """
    model_set, checkpoint_set = set(model_keys), set(checkpoint_keys)
    return sorted(model_set - checkpoint_set), sorted(checkpoint_set -
    model_set)
```

Two set differences give the two lists. Missing keys are what the model needs and the checkpoint lacks. Unexpected keys are the reverse.

B26-L1 exercise 5

Hints. (1) Find keys that do not start with the prefix first. (2) Raise if there are any. (3) Build a new dict with `k.removeprefix(prefix)`.

Answer.

```python
def strip_prefix(state: dict[str, object], prefix: str) -> dict[str, object]:
    """Remove a wrapper prefix from every checkpoint key.

    Args:
        state: A state_dict, or any dictionary keyed by parameter name.
        prefix: The prefix every key must start with, such as "model.".

    Returns:
        A new dictionary with the prefix removed from each key.

    Raises:
        ValueError: If any key does not start with prefix.
    """
    wrong = [k for k in state if not k.startswith(prefix)]
    if wrong:
        raise ValueError(f"keys without prefix {prefix!r}: {wrong[:3]}")
    return {k.removeprefix(prefix): v for k, v in state.items()}
```

`removeprefix` touches only the start of the key, unlike `replace`. The guard turns a wrong prefix into an error instead of a silent load of nothing.

B26-L1 exercise 6

Hints. (1) Do the checkpoint's names match the model's? (2) What does strict=False do with names that do not match? (3) Which line throws away the report?

Answer. The bug is on line 7.

The keys are `model.encoder...` but the model expects `encoder...`, so nothing matches. `load_state_dict(strict=False)` reports every key and raises nothing, and line 7 discards that report. Strip the prefix and check `missing_keys` and `unexpected_keys`.

Common wrong answers.

- Line 6: The filter keeps the encoder keys. They still carry the model. prefix, but the line that should notice that is the load.
- Line 9: Freezing the encoder is a valid strategy. The encoder it freezes is still random.
- Line 5: torch.load reads the file correctly. The weights fail to arrive later.

B26-L2 Learning rates for fine-tuning, warm-up and decay

B26-L2 exercise 1

Hints. (1) What is random at the start of fine-tuning? (2) What happens to good weights after a large, noisy step? (3) Keep the first steps small.

Answer. a) Early gradients from the random head are large and noisy, and full-size steps then can damage the pretrained weights Because: r1) Small first steps let the head and the optimiser's averages settle before the weights move much.

The new head starts at random and AdamW's estimates start from few samples, so the first gradients are unreliable. Warm-up keeps those steps small until training settles.

Common wrong answers.

- `b`: The tokenizer comes ready with the pretrained model. Warm-up is about the size of weight updates.
- `c`: Steps take the same time at any rate. A rate near 0 moves the weights less.
- `d`: Warm-up changes training only. You still need validation data to choose and check the model.
- `a|r2`: Right answer, wrong reason. There is no threshold. Any rate above 0 moves the weights, which is why the first steps are kept small.

B26-L2 exercise 2

Hints. (1) Steps 0, 2, 4, 6, 8. (2) Warm-up ends at step 2. (3) From step 2, divide the steps left by 6.

Answer.

```
1 [0.0, 1.0, 0.67, 0.33, 0.0]
```

At step 2 warm-up is complete, so the factor is 1. After that it is `(8 - step) / 6`: 4/6, 2/6 and 0.

Common wrong answers.

- `[0.0, 0.25, 0.5, 0.75, 1.0]`: That is warm-up all the way. Warm-up ends at step 2, then the factor decays.
- `[0.0, 1.0, 0.75, 0.5, 0.25]`: Decay runs over the 6 steps from 2 to 8, so it falls by 1/6 a step and reaches 0 at step 8.

B26-L2 exercise 3

Hints. (1) How many batches does the whole run take? (2) The scheduler moves one step per batch. (3) `epochs` and `step`.

Answer.

```
total_steps = len(loader) * epochs
for epoch in range(epochs):
    for xb, yb in loader:
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        scheduler.step()
```

`len(loader)` is the number of batches per epoch, so times `epochs` gives every optimiser step. `scheduler.step()` follows `optimizer.step()` inside the batch loop.

Common wrong answers.

- g1 = `1`; g2 = `step`: That sizes the schedule for one epoch, so the rate hits 0 after the first epoch.

B26-L2 exercise 4

Hints. (1) Check the arguments first. (2) Below warm-up: `step / warmup_steps`. (3) Then `max(0.0, (total_steps - step) / (total_steps - warmup_steps))`.

Answer.

```
def warmup_linear(step: int, warmup_steps: int, total_steps: int) -> float:

    """Learning-rate factor: linear warm-up from 0 to 1, then linear decay to 0.

     Args:
         step: Optimiser steps taken so far, from 0.
         warmup_steps: Steps to rise from 0 to 1.
         total_steps: Step at which the factor reaches 0.

     Returns:
         The factor in [0, 1].

     Raises:
         ValueError: If warmup_steps is negative or not below total_steps.
     """
     if warmup_steps < 0 or warmup_steps >= total_steps:
         raise ValueError(f"need 0 <= warmup_steps < total_steps, got {warmup_steps}, {total_steps}")
```

```
    if step < warmup_steps:
        return step / warmup_steps
    return max(0.0, (total_steps - step) / (total_steps - warmup_steps))
```

The two straight lines meet at 1 at the end of warm-up. `max(0.0, ...)` keeps the factor at 0 if training runs past `total_steps`, where a negative rate would push the weights uphill.

B26-L2 exercise 5

Hints. (1) Guard the counts and the share first. (2) `math.ceil(n_examples / batch_size)` batches per epoch. (3) Multiply by epochs, then take `int(warmup_share * total)`.

Answer.

```
import math


def schedule_steps(n_examples: int, batch_size: int, epochs: int, warmup_share: float) -> tuple[int, int]:
    """Total optimiser steps and warm-up steps for a run.

    Args:
        n_examples: Training examples.
        batch_size: Examples per batch; the last batch may be smaller.
        epochs: Passes over the data.
        warmup_share: Share of all steps spent warming up, in [0, 1).

    Returns:
        (total_steps, warmup_steps).

    Raises:
        ValueError: If a count is below 1 or warmup_share is outside [0, 1).
    """
    if min(n_examples, batch_size, epochs) < 1:
        raise ValueError("n_examples, batch_size and epochs must be at least 1")
    if not 0.0 <= warmup_share < 1.0:
        raise ValueError(f"warmup_share must be in [0, 1), got {warmup_share}")
    total = math.ceil(n_examples / batch_size) * epochs
    return total, int(warmup_share * total)
```

A DataLoader without `drop_last` yields a final, smaller batch, and the optimiser steps on it too. Rounding down leaves the schedule short, so the rate reaches 0 before training ends.

B26-L2 exercise 6

Hints. (1) How many times does scheduler.step() run in total? (2) The schedule expects 250 steps. (3) Look at the indentation of the last line.

Answer. The bug is on line 10.

`scheduler.step()` sits in the epoch loop, so it runs 5 times in a schedule sized for 250 steps, and the 25-step warm-up never finishes. Indent it into the batch loop, after `optimizer.step()`.

Common wrong answers.

- Line 1: 250 steps is the right total for 5 epochs of 50 batches. The scheduler never gets there.
- Line 2: A 10% warm-up of 25 steps is sensible. How often does the schedule move?
- Line 9: The optimiser steps once per batch, as it should.

B26-L3 LoRA, fine-tuning with a low-rank update

B26-L3 exercise 1

Hints. (1) What is B @ A when B is zero? (2) What does the adapted layer compute then? (3) Compare it with the pretrained layer.

Answer. a) So B @ A is zero and the adapted model starts identical to the pretrained one Because: r1) Training then starts from the pretrained model's behaviour and moves away only as far as the data asks.

With `B` at zero the update is zero, so the first forward pass equals the pretrained model's. Training starts from known-good behaviour.

Common wrong answers.

- `b`: B changes from the first step, so the update gains up to rank r as training goes on.
- `c`: B is still trained. Its starting value does not change how many values train.
- `d`: A gets no gradient at the first step, while B is zero. After B moves, both train.
- `a|r2`: Right answer, wrong reason. A zero matrix takes as much memory as any other. The point is the starting behaviour.

B26-L3 exercise 2

Hints. (1) A has r * d_in values, B has d_out * r. (2) W has d_out * d_in. (3) What does a zero B add?

Answer.

```
20 24 True
```

`A` and `B` hold 12 and 8 values, 20 in all, against 24 in `W`. At these tiny sizes LoRA barely saves anything. At 768 by 768 with `r = 8` it trains 2%. With `B` at zero the outputs match.

Common wrong answers.

- `20 24 False`: B is zero, so the update adds nothing and the outputs match.
- `48 24 True`: A is 2 by 6 and B is 4 by 2: 12 plus 8 values.

B26-L3 exercise 3

Hints. (1) The scale is alpha over the rank. (2) Which product has W's shape? (3) `r` and `B`.

Answer.

```
r = A.shape[0]
W_merged = W + (alpha / r) * B @ A
```

`B @ A` is `(d_out, r)` times `(r, d_in)`, the shape of `W`. Scaling by `alpha / r` matches the forward pass, so the merged layer gives the same outputs.

Common wrong answers.

- g1 = `r`; g2 = `A.T`: A.T @ A has shape (d_in, d_in). The update B @ A has W's shape, (d_out, d_in).

B26-L3 exercise 4

Hints. (1) `r = A.shape[0]`. (2) Check `A.shape[1] == W.shape[1]` and `B.shape == (W.shape[0], r)`. (3) Add `(alpha / r) * (x @ A.T @ B.T)` to `x @ W.T`.

Answer.

```
import numpy as np


def lora_forward(x: np.ndarray, W: np.ndarray, A: np.ndarray, B:
    np.ndarray, alpha: float) -> np.ndarray:
    """Frozen layer plus the scaled low-rank update.

    Args:
        x: Inputs, shape (n, d_in).
        W: Frozen weight, shape (d_out, d_in).
        A: Down projection, shape (r, d_in).
```

```
        B: Up projection, shape (d_out, r).
        alpha: Scale numerator; the update is multiplied by alpha / r.

    Returns:
        Outputs, shape (n, d_out).

    Raises:
        ValueError: If A's width differs from W's or B is not (d_out, r).
    """
    r = A.shape[0]
    if A.shape[1] != W.shape[1] or B.shape != (W.shape[0], r):
        raise ValueError(f"shapes do not fit: W {W.shape}, A {A.shape}, B
{B.shape}")
    return x @ W.T + (alpha / r) * (x @ A.T @ B.T)
```

The frozen path is `x @ W.T`. The update goes down to `r` values with `A`, back up to `d_out` with `B`, and is scaled by `alpha / r`. The shape checks catch a transposed `B`, which a square layer would otherwise accept.

B26-L3 exercise 5

Hints. (1) `r = A.shape[0]`. (2) `B @ A` has W's shape. (3) `W + (alpha / r) * B @ A` makes a new array; `+=` would not.

Answer.

```
import numpy as np


def merge_lora(W: np.ndarray, A: np.ndarray, B: np.ndarray, alpha: float)
-> np.ndarray:
    """One weight matrix equal to the frozen layer plus its LoRA update.

    Args:
        W: Frozen weight, shape (d_out, d_in).
        A: Shape (r, d_in).
        B: Shape (d_out, r).
        alpha: Scale numerator.

    Returns:
        A new array of W's shape.
    """
    r = A.shape[0]
    return W + (alpha / r) * B @ A
```

Merging adds the scaled update to a copy of `W`. Changing `W` in place would destroy the frozen weight that other adapters share.

B26-L3 exercise 6

Hints. (1) What scale does the forward pass use? (2) With r = 4 and alpha = 8, what should the scale be? (3) 2, not 32.

Answer. The bug is on line 6.

Fixed version:

```
1 import numpy as np
2
3
4 def merged_weight(W, A, B, alpha):
5     r = A.shape[0]
6     scaling = alpha / r
7     update = B @ A
8     return W + scaling * update
```

The forward pass scales the update by `alpha / r`, here 2. Line 6 multiplies instead, giving 32, so the merged layer applies an update 16 times too large.

Common wrong answers.

- Line 7: B @ A has W's shape and is the right product. Look at how it is scaled.
- Line 5: r is the number of rows of A, which is right.

B26-L4 Fine-tuning a text classifier with Hugging Face

B26-L4 exercise 1

Hints. (1) Did the pretrained checkpoint have a 4-class head? (2) Which keys are listed? (3) A new head starts at random.

Answer. a) The new 4-class head starts at random, as expected. Fine-tuning trains it Because: r1) The checkpoint was saved without a 4-class head, so the head has nothing to load from.

A pretrained encoder has no head for your 4 classes, so the head is created at random and reported as missing. That is the part fine-tuning must train first.

Common wrong answers.

- `b`: The encoder loaded. Only the head, which the checkpoint never had, is missing.
- `c`: num_labels is your task's number of classes. The head is new whatever the number.
- `d`: Only the keys listed are missing. Encoder keys are not in the list, so they loaded.
- `a|r2`: Right answer, wrong reason. Nothing is reset. The checkpoint has no head of this shape to load.

B26-L4 exercise 2

Hints. (1) What type are the labels? (2) Float labels with num_labels above 1 give which problem type? (3) One text, three classes.

Answer.

```
multi_label_classification (1, 3)
```

The model sets `problem_type` from the first labels it sees. Floats with 3 labels mean multi-label, so it trains with a sigmoid per class. Pass integer class IDs, such as `torch.tensor([1])`, for one label per text.

Common wrong answers.

- `single_label_classification (1, 3)`: The labels are floats and num_labels is 3, so the model picks the multi-label loss.
- `multi_label_classification (1, 4, 3)`: Sequence classification gives one row of logits per text, not per token.

B26-L4 exercise 3

Hints. (1) Texts differ in length. (2) Which framework is the model in? (3) `True` and `pt`.

Answer.

```
batch = tokenizer(texts, padding=True, truncation=True, max_length=128,
    return_tensors="pt")
```

`padding=True` pads to the longest text in the batch and sets `attention_mask` to 0 there. `return_tensors="pt"` returns PyTorch tensors.

Common wrong answers.

- g1 = `False`; g2 = `pt`: Without padding, texts of different lengths cannot stack into one tensor.
- g1 = `True`; g2 = `np`: "np" gives NumPy arrays. The PyTorch model needs "pt".

B26-L4 exercise 4

Hints. (1) `sorted(set(labels))`. (2) Raise if fewer than 2 remain. (3) `dict(enumerate(names))`, then invert it.

Answer.

```
def label_maps(labels: list[str]) -> tuple[dict[int, str], dict[str,
    int]]:
    """ID-to-label and label-to-ID maps for a classifier head.

    Args:
        labels: One label per row, repeats allowed.
```

```
    Returns:
        (id2label, label2id), numbering the sorted distinct labels from 0.

    Raises:
        ValueError: If there are fewer than 2 distinct labels.
    """
    names = sorted(set(labels))
    if len(names) < 2:
        raise ValueError(f"need at least 2 distinct labels, got {names}")
    id2label = dict(enumerate(names))
    return id2label, {name: i for i, name in id2label.items()}
```

Sorting fixes the IDs, so a model trained on one extract and scored on another reads class 1 as the same label. A single class cannot train a classifier, so the guard stops it early.

B26-L4 exercise 5

Hints. (1) Cut first: `s[:max_length]`. (2) The width is the longest cut sequence. (3) Pad with `[pad_id] * (width - len(s))` and build the mask the same way.

Answer.

```
def pad_batch(
    seqs: list[list[int]], pad_id: int, max_length: int
) -> tuple[list[list[int]], list[list[int]]]:
    """Truncate and right-pad token IDs, with an attention mask.

    Args:
        seqs: Token IDs per text.
        pad_id: ID used for padding.
        max_length: Longest allowed sequence.

    Returns:
        (input_ids, attention_mask), all rows of equal length.

    Raises:
        ValueError: If seqs is empty or max_length is below 1.
    """
    if not seqs or max_length < 1:
        raise ValueError("need at least one sequence and max_length of at least 1")
    cut = [s[:max_length] for s in seqs]
    width = max(len(s) for s in cut)
    ids = [s + [pad_id] * (width - len(s)) for s in cut]
    mask = [[1] * len(s) + [0] * (width - len(s)) for s in cut]
    return ids, mask
```

Truncation runs first, then padding to the longest remaining sequence, so no row is wider than needed. The mask tells attention which positions are real.

B26-L4 exercise 6

Hints. (1) Which parameters does peft leave trainable? (2) What tells peft that this is sequence classification? (3) task_type.

Answer. The bug is on line 7.

Without `task_type=TaskType.SEQ_CLS`, peft freezes everything except the LoRA matrices, including the new classifier. The head stays at its random start, so the model cannot map features to classes. Add the task type.

Common wrong answers.

- Line 6: Loading with num_labels creates a new head, as it should. The head then needs to be trainable.
- Line 8: get_peft_model applies the config it is given. The config is what lacks something.
- Line 9: Train mode is right. It turns dropout on but does not decide what trains.

B26-L5 Token classification, named entities and subword labels

B26-L5 exercise 1

Hints. (1) Which loss argument has a default of -100? (2) What happens to an ignored position's loss? (3) Count words, not pieces.

Answer. a) Cross-entropy ignores -100 by default, so those positions add nothing to the loss Because: r1) Each word then counts once in the loss, whatever its number of tokens.

`nn.CrossEntropyLoss` has `ignore_index=-100` by default. Positions with that label are skipped, so special tokens do not count and a long word does not count several times.

Common wrong answers.

- `b`: O is a real class with its own ID, usually 0. -100 means no label at all.
- `c`: Labels never reach the tokenizer. They go to the loss.
- `d`: The model still outputs logits at every position. You choose to read only each word's first token.
- `a|r2`: Right answer, wrong reason. Ignored positions give no gradient at all, not a smaller one.

B26-L5 exercise 2

Hints. (1) None gives -100. (2) Word 1 has two tokens. (3) Only the first token of each word carries the label.

Answer.

```
[-100, 1, 0, -100, 3, -100]
```

`[CLS]` and `[SEP]` get -100. Words 0, 1 and 2 put their labels on their first tokens, and the second piece of word 1 gets -100.

Common wrong answers.

- `[-100, 1, 0, 0, 3, -100]`: The second piece of word 1 repeats the previous word ID, so it gets -100.
- `[1, 0, 3, -100, -100, -100]`: Labels follow word_ids, not list position. The first token is special.

B26-L5 exercise 3

Hints. (1) What does word_ids() give for [CLS] and [SEP]? (2) Which label does the loss ignore? (3) `None` and `-100`.

Answer.

```
for word_id in enc.word_ids():
    if word_id is None:
        labels.append(-100)
```

Special tokens have no word, so `word_ids()` gives `None`. They get `-100`, which the loss skips.

Common wrong answers.

- g1 = `None`; g2 = `0`: 0 is usually the O tag, a real class. Special tokens need the ignored label, -100.

B26-L5 exercise 4

Hints. (1) Keep the previous word ID. (2) `is None` first, then compare with the previous ID. (3) Check the ID against `len(word_labels)` before indexing.

Answer.

```
def align_labels(word_ids: list[int | None], word_labels: list[int]) ->
    list[int]:
    """Token labels from word labels, first token of each word only.

    Args:
        word_ids: Word index per token, None for special tokens.
        word_labels: One label ID per word.

    Returns:
```

```
        One label per token, -100 where the loss should ignore it.

    Raises:
        ValueError: If a word index has no label.
    """
    aligned: list[int] = []
    previous: int | None = None
    for word_id in word_ids:
        if word_id is None:
            aligned.append(-100)
        elif word_id != previous:
            if word_id >= len(word_labels):
                raise ValueError(f"word {word_id} has no label")
            aligned.append(word_labels[word_id])
        else:
            aligned.append(-100)
        previous = word_id
    return aligned
```

Tracking the previous word ID separates a word's first token from its later pieces. `is None` matters: word 0 is falsy, and a truthiness test would give the first word of every sentence -100.

B26-L5 exercise 5

Hints. (1) Check the lengths first. (2) Walk both lists together with `zip`. (3) Keep a prediction when the word ID is not None and differs from the previous one.

Answer.

```
def word_predictions(word_ids: list[int | None], token_preds: list[int]) -> list[int]:
    """One label per word, from the word's first token.

    Args:
        word_ids: Word index per token, None for special tokens.
        token_preds: Predicted label per token.

    Returns:
        Predicted labels in word order.

    Raises:
        ValueError: If the lists differ in length.
    """
    if len(word_ids) != len(token_preds):
        raise ValueError(f"{len(word_ids)} word IDs but {len(token_preds)} predictions")
    preds: list[int] = []
    previous: int | None = None
    for word_id, pred in zip(word_ids, token_preds):
        if word_id is not None and word_id != previous:
```

```
            preds.append(pred)
        previous = word_id
    return preds
```

Training put each word's label on its first token, so prediction reads the same position. Later pieces were never trained against a label and are less reliable.

B26-L5 exercise 6

Hints. (1) What is the word ID of the first word? (2) Is 0 truthy? (3) Test for None with `is None`.

Answer. The bug is on line 4.

Fixed version:

```
def align_labels(word_ids, word_labels):
    aligned, previous = [], None
    for word_id in word_ids:
        if word_id is None:
            aligned.append(-100)
        elif word_id != previous:
            aligned.append(word_labels[word_id])
        else:
            aligned.append(-100)
        previous = word_id
    return aligned
```

Word 0 is falsy, so `if not word_id` treats it like a special token and gives it -100. The first word of every sentence never trains. `if word_id is None` separates the two.

Common wrong answers.

- Line 6: Comparing with the previous ID is how a new word is spotted. Word 0 never reaches this line.
- Line 10: Tracking the previous ID is right. The first word is lost earlier.

B26-L6 Evaluating a fine-tuned model

B26-L6 exercise 1

Hints. (1) What does predicting O everywhere score? (2) Which measure counts whole entities? (3) Compare with the baseline.

Answer. a) Nothing yet: tagging every token O would also score 0.90. Compute entity-level F1 Because: r1) The score matches what the all-O baseline gets, so it says nothing about entities.

When 90% of tokens are `O`, an all-`O` tagger scores 0.90 token accuracy and 0 entity F1. Only an entity-level score shows whether the model finds entities.

Common wrong answers.

- `b`: Token accuracy counts tokens, mostly O tokens, not entities.
- `c`: The all-O baseline also scores 0.90, so 0.90 shows no gain over it.
- `d`: It might find some. Token accuracy cannot tell you either way.
- `a|r2`: Right answer, wrong reason. There is no fixed gap between the two measures.

B26-L6 exercise 2

Hints. (1) End positions are exclusive. (2) I-LOC after B-LOC continues the entity. (3) What does the added O do?

Answer.

```
1 [('PER', 0, 1), ('LOC', 2, 4), ('ORG', 5, 6)]
```

Each span closes at the first tag that does not continue it. The trailing `"O"` closes `ORG`, which would otherwise be lost.

Common wrong answers.

- `[('PER', 0, 1), ('LOC', 2, 3), ('LOC', 3, 4), ('ORG', 5, 6)]`: I-LOC continues the LOC that B-LOC started, so it is one span from 2 to 4.
- `[('PER', 0, 1), ('LOC', 2, 4)]`: The extra O at the end closes the ORG span that runs to the last word.

B26-L6 exercise 3

Hints. (1) The test set waits until the end. (2) You need the baselines' scores before you tune. (3) Check the spread before the final score.

Answer.

1. Split into training, validation and test sets, and set the test set aside
2. Score the majority-class baseline and a simple model on validation
3. Fine-tune and choose strategy, learning rate and epochs on validation
4. Rerun the chosen set-up with three seeds and record mean and range
5. Score the final set-up once on the test set

Split first, so nothing leaks. Baselines set the bar, tuning and seeds use validation only, and the test set gives one final, unbiased number.

Common wrong answers.

- Baselines come before tuning, on the same validation data, so you know what you must beat.
- Seeds belong to the choice of set-up. Run them on validation, then look at the test set once.

B26-L6 exercise 4

Hints. (1) Add an "O" to the end so the last entity closes. (2) A tag continues the entity only if it is I- with the same type. (3) Otherwise close any open span, then start one unless the tag is O.

Answer.

```python
def bio_spans(tags: list[str]) -> set[tuple[str, int, int]]:
    """Entity spans (type, start, end exclusive) from BIO tags.

    Args:
        tags: One BIO tag per word, such as "B-LOC", "I-LOC" or "O".

    Returns:
        The set of spans.
    """
    spans: set[tuple[str, int, int]] = set()
    start: int | None = None
    kind: str | None = None
    for i, tag in enumerate([*tags, "O"]):
        continues = tag.startswith("I-") and tag[2:] == kind
        if not continues:
            if start is not None and kind is not None:
                spans.add((kind, start, i))
            start, kind = (i, tag[2:]) if tag != "O" else (None, None)
    return spans
```

Every tag either continues the open entity or closes it. Appending `"O"` closes the last one. Comparing types stops `B-LOC I-PER` from becoming one mixed entity.

B26-L6 exercise 5

Hints. (1) Check lengths, then build both span sets. (2) Correct spans: `len(true & pred)`. (3) Guard each division and the F1 denominator.

Answer.

```python
def entity_f1(true_tags: list[str], pred_tags: list[str]) -> float:
    """F1 over whole entities: type, start and end must all match.

    Args:
        true_tags: Gold BIO tags.
        pred_tags: Predicted BIO tags, same length.

    Returns:
        Entity-level F1 in [0, 1].

    Raises:
        ValueError: If the lists differ in length.
    """
    if len(true_tags) != len(pred_tags):
```

```
        raise ValueError(f"{len(true_tags)} true tags but {len(pred_tags)} predicted")
    true, pred = bio_spans(true_tags), bio_spans(pred_tags)
    if not true and not pred:
        return 1.0
    correct = len(true & pred)
    precision = correct / len(pred) if pred else 0.0
    recall = correct / len(true) if true else 0.0
    return 0.0 if precision + recall == 0 else 2 * precision * recall / (precision + recall)
```

Spans compare whole entities, so a span cut short counts as one miss and one false alarm. The guards cover the cases with no entities on one or both sides, which are common on short survey answers.

B26-L6 exercise 6

Hints. (1) Most words in an answer are not brands. (2) What would a team without a model use? (3) One seed or the best seed both mislead.

Answer.

- Entity-level F1, overall and for each type, because most tokens are O.
- A brand-list lookup is the cheap rival to beat, and full fine-tuning shows what LoRA gives up. Compare all of them on validation.
- Three seeds with mean and range, then one final score on the test set.

Entity F1 measures what matters, baselines show whether the model is worth its cost, and several seeds show how much of the score is luck.

Common wrong answers.

- metric = `token`: Most tokens are O, so token accuracy stays high even when brands are missed.
- metric = `loss`: Training loss says how well the model fits its training data, not how it does on new answers.
- compare = `none`: Without a baseline you cannot tell whether a simple lookup would do as well.
- compare = `test`: Choosing between set-ups on the test set makes it validation data. Compare on validation.
- seeds = `best`: The best of five reports luck. Report the mean and the range.
- seeds = `one`: On 800 answers, one seed can be far from typical. In the lab, LoRA's seeds ranged from 0.929 to 0.990.